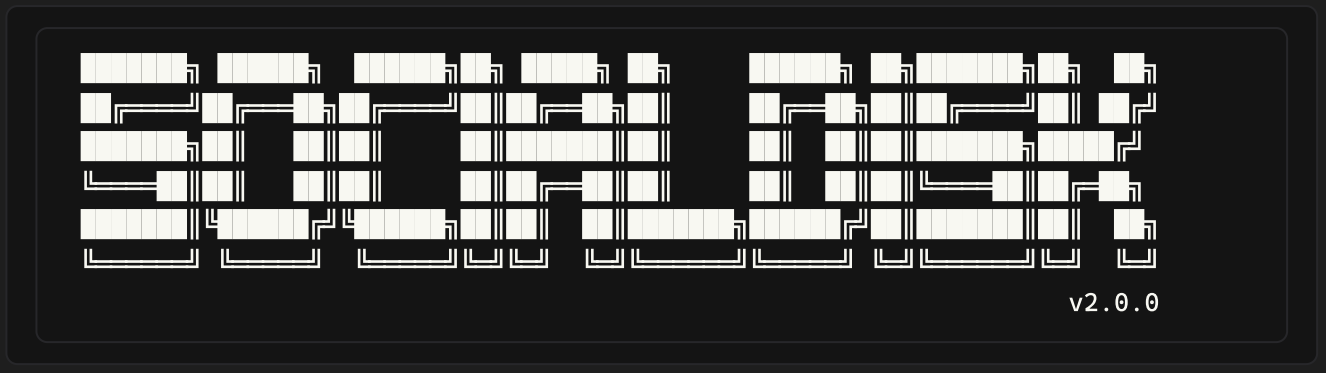
SoclaDisk اداة





Hide & encrypt data inside images, video, and audio

Survives compression on Instagram, WhatsApp, TikTok, YouTube, and 10+ more

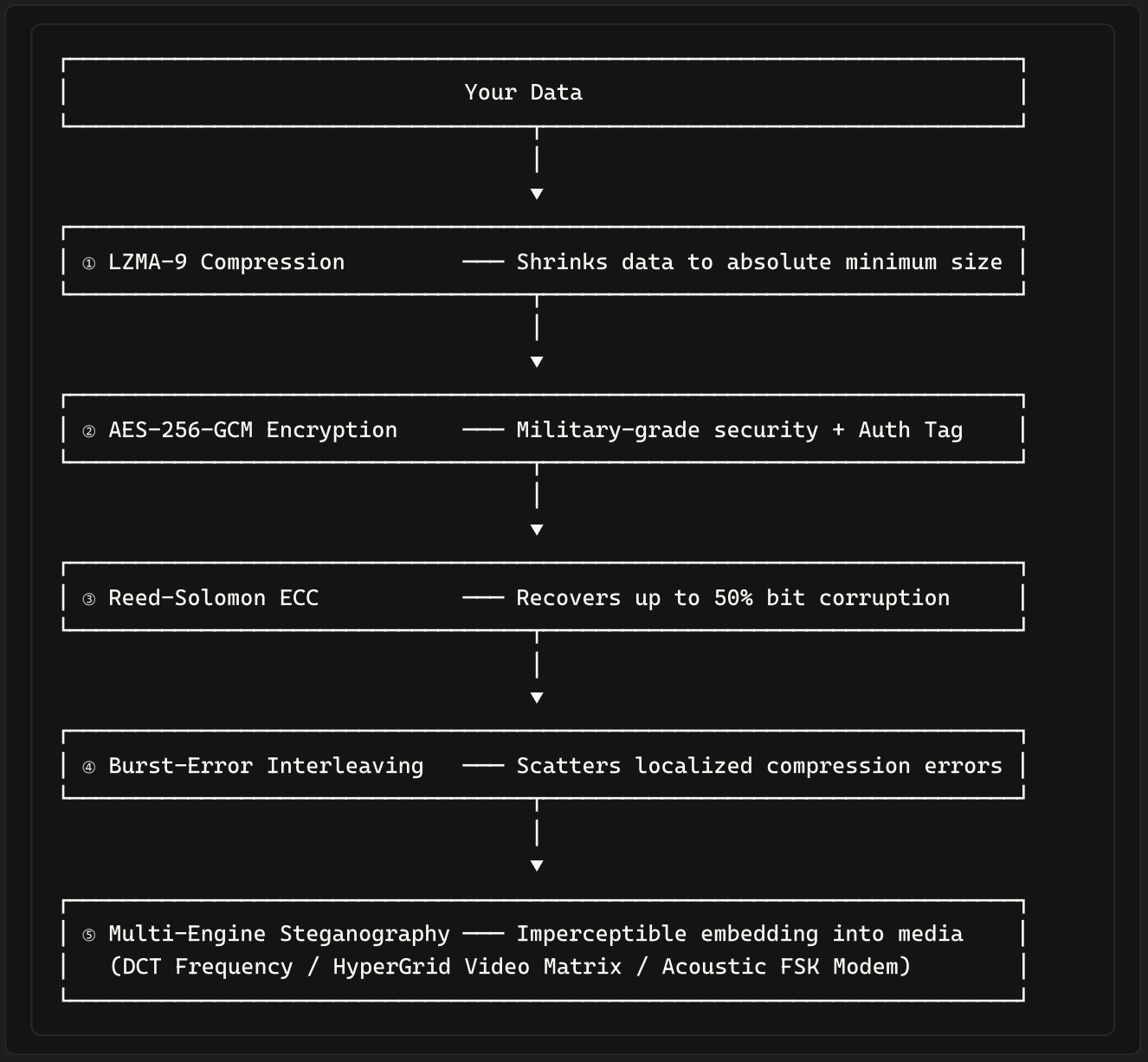
GB English · SA العربية

GB English

?What is SocialDisk 📖

SocialDisk is a multi-platform steganography suite that lets you **secretly hide any file or message** inside ordinary media files — images, videos, or audio — that you can then post on any social media platform. The person on the other end downloads the media, runs SocialDisk, and recovers the hidden data perfectly, even after the platform's compression.

> Every byte of hidden data is protected by a **5-layer security pipeline** before being embedded:


Your Data
① LZMA-9 Compression —— Shrinks data to absolute minimum size
② AES-256-GCM Encryption —— Military-grade security + Auth Tag
③ Reed-Solomon ECC —— Recovers up to 50% bit corruption
④ Burst-Error Interleaving —— Scatters localized compression errors
⑤ Multi-Engine Steganography —— Imperceptible embedding into media
(DCT Frequency / HyperGrid Video Matrix / Acoustic FSK Modem)

Features ✨

#	Feature	Description
🔒	**AES-256-GCM Encryption**	Military-grade authenticated encryption. Any tampering is detected and rejected.
🧬	**Reed-Solomon ECC**	Error-correcting code that can recover data even when up to 50% of bits are damaged by compression.
🗜️	**LZMA-9 Compression**	Maximum-preset LZMA compression applied before encryption — shrinks payload to minimum.
🔀	**Burst-Error Interleaving**	Shuffles bytes pseudo-randomly (password-seeded) so compression artifacts are spread across the ECC blocks, maximizing recovery.
🔑	**PBKDF2-SHA256 Key Derivation**	Password is never used directly — stretched with 100,000 iterations + random 16-byte salt.
📸	**DCT Image Engine**	Modifies JPEG frequency-domain coefficients. Changes are invisible to the human eye and survive re-compression.
🎬	**HyperGrid Video Engine**	Encodes data as a visual pixel grid in video frames. Survives H.264/H.265 video codec compression.
🎵	**Acoustic FSK Audio Engine**	Converts data to audio tones (Frequency Shift Keying). Tones survive AAC and Opus compression at any bitrate.
🌍	**14+ Platform Profiles**	Each platform has a pre-tuned profile: compression strength, ECC level, redundancy factor, frequencies.
🖥️	**Beautiful Interactive CLI**	Full-screen menu with colors, tables, and progress bars (powered by Rich library).
⌨️	**Command-Line Mode**	Fully scriptable — integrate in pipelines, automation, or other tools.
🐍	**Python API**	Clean `SocialVault` class for direct integration in your own Python code.
✅	**100% Test Coverage**	5-engine test suite with real JPEG/video/audio compression simulation.

Supported Platforms 🌍

Platform	🖼️ Image	🎬 Video	🎵 Audio	Notes
Instagram	✅	✅	✅	JPEG q=70, H.264
WhatsApp	✅	✅	✅	Most aggressive compression — highest ECC
TikTok	✅	✅	✅	Video-first platform
YouTube	—	✅	✅	H.264/H.265 video
Twitter / X	✅	✅	—	JPEG + MP4
Discord	✅	✅	✅	Lightest compression — highest capacity
Telegram	✅	✅	✅	Lossless option available
Facebook	✅	✅	—	JPEG q=85
LinkedIn	✅	—	—	Image posts
Snapchat	✅	✅	—	JPEG + MP4
Signal	✅	—	✅	Privacy-focused
WeChat	✅	—	—	JPEG compression
Reddit	✅	—	—	PNG/JPEG
iMessage	✅	—	✅	HEIC/JPEG + audio

Installation 🚀

Requirements

- Python **3.9** or higher
- pip

Install

```
# Clone the repository
git clone https://github.com/YOUR_USERNAME/SocialDisk.git
cd SocialDisk

# Install dependencies
pip install -r requirements.txt
```

Dependencies installed automatically

```
numpy       – Array operations, FFT signal processing
scipy       – WAV file I/O, audio resampling
opencv-python – Image and video processing (JPEG/MP4)
cryptography  – AES-256-GCM encryption
reedsolo    – Reed-Solomon error correction codec
rich        – Beautiful terminal interface
```

Verify installation

```
python test_import.py    # Should print: All imports OK ✅
python full_test.py      # Should show:  5/5 tests passed ✅
```

Usage — Interactive Mode (Recommended for beginners) 🖥️

Just run with no arguments — a beautiful full-screen menu will appear:

```
python cli.py
```

The menu lets you:

- Choose the operation (Hide / Extract / List platforms)
- Select the platform (Instagram, WhatsApp, etc.)
- Select the media type (Image / Video / Audio)
- Browse for files
- Set password

- See results with progress bars

Usage — Command-Line Mode ⌨️

Hide data inside an image

```
python cli.py hide \
  --platform instagram \
  --type image \
  --input  secret.txt \
  --cover  photo.jpg \
  --output stego.jpg \
  --password "my_strong_password"
```

Extract data from an image

```
python cli.py extract \
  --platform instagram \
  --type image \
  --input  stego_downloaded.jpg \
  --output recovered.txt \
  --password "my_strong_password"
```

Hide data inside a video

```
python cli.py hide \
  --platform whatsapp \
  --type video \
  --input  document.pdf \
  --cover  clip.mp4 \
  --output stego_clip.mp4 \
  --password "my_strong_password"
```

Hide data inside an audio file

```
python cli.py hide \
  --platform discord \
  --type audio \
  --input  message.txt \
  --output encoded_audio.wav \
  --password "my_strong_password"
```

Extract from audio

```
python cli.py extract \
  --platform discord \
  --type audio \
  --input  encoded_audio.wav \
  --output decoded_message.txt \
  --password "my_strong_password"
```

List all supported platforms with details

```
python cli.py --platforms
```

Show full help

```
python cli.py --help
```

Python API

Quick example — Image

```
from socialdisk import SocialVault

# Create a vault for Instagram
vault = SocialVault("instagram")

# Hide a file inside an image
stats = vault.hide_in_image(
    cover_path="photo.jpg",
    output_path="stego.jpg",
    data=open("secret.pdf", "rb").read(),
    password="my_strong_password"
)
print(f"Embedded {stats['capacity_used_percent']}% of image capacity")

# Extract the file (after downloading from Instagram)
recovered = vault.recover_from_image(
    stego_path="downloaded_from_instagram.jpg",
    password="my_strong_password"
)
open("recovered.pdf", "wb").write(recovered)
```

Quick example — Video

```
from socialdisk import SocialVault

vault = SocialVault("whatsapp")

# Embed a file into a video
vault.file_to_video(
    output_video="stego.mp4",
    data=open("secret.zip", "rb").read(),
    password="my_strong_password"
)

# Recover from video
recovered = vault.video_to_file(
    video_path="downloaded_from_whatsapp.mp4",
    password="my_strong_password"
)
```

Quick example — Audio

```
from socialdisk import SocialVault

vault = SocialVault("discord")

# Encode message into WAV audio
vault.text_to_audio(
    output_wav="encoded.wav",
    text="My secret message",
    password="my_strong_password"
)

# Decode from audio
text = vault.audio_to_text(
    audio_path="encoded.wav",
    password="my_strong_password"
)
print(text)  # "My secret message"
```

Low-level API — Direct engine access

```
from socialdisk.engines.image import DCTImageEngine
from socialdisk.engines.video import HyperGridVideoEngine
from socialdisk.engines.audio import AcousticFSKEngine
from socialdisk.platforms import get_platform

# Get platform profile
profile = get_platform("instagram")

# Image engine
stats = DCTImageEngine.embed("cover.jpg", "stego.jpg", b"secret", "password",
profile=profile.image)
data  = DCTImageEngine.extract("stego.jpg", "password", profile=profile.image)

# Video engine
stats = HyperGridVideoEngine.encode("stego.mp4", b"secret", "password")
data  = HyperGridVideoEngine.decode("stego.mp4", "password")

# Audio engine
stats = AcousticFSKEngine.encode_wav("encoded.wav", b"secret", "password")
data  = AcousticFSKEngine.decode_media("encoded.wav", "password")
```

List all platforms programmatically

```
from socialdisk.platforms import list_platforms, get_platform

platforms = list_platforms()
print(platforms)
# ['instagram', 'whatsapp', 'tiktok', 'youtube', 'twitter', ...]

profile = get_platform("instagram")
print(profile.image.delta)         # DCT delta strength
print(profile.image.ecc_symbols)   # Reed-Solomon symbols
print(profile.audio.freq_zero)     # FSK frequency for bit=0
```

Technical Deep Dive 🔬

Image Engine — DCT Steganography

The DCT (Discrete Cosine Transform) engine works in the frequency domain of JPEG images:

1. Split image into 8×8 pixel blocks
2. Apply DCT to convert spatial data to frequency coefficients
3. Modify mid-frequency coefficients by `±delta` based on data bits
4. Apply inverse DCT — the change is visually imperceptible
5. JPEG re-compression at any quality ≥ platform threshold preserves the modification

Why it survives JPEG re-compression:

Mid-frequency coefficients survive JPEG quantization because they are quantized to coarse values, and our modification (`delta`) is large enough to survive the quantization step, but small enough to be invisible.

Platform-specific parameters:

Platform	Delta	ECC Symbols	Redundancy
Discord	22	48	3×
Instagram	50	80	3×
WhatsApp	55	96	5×
Signal	60	100	5×

Video Engine — HyperGrid

The HyperGrid engine encodes data as a visible-but-noise-like pixel grid in video frames:

1. Pack data bits into an 8-bit grid pattern
2. Assign grid pixels to a specific region of specific frames
3. Use high-contrast pixel values (black/white blocks) so the signal survives H.264 lossy compression
4. Reed-Solomon + interleaving protect against block artifacts

---

Audio Engine — Acoustic FSK

Frequency Shift Keying (FSK) encodes each data bit as an audio tone:

```
Frame structure:
  [Silence 0.25s] [SYNC preamble 0.5s] [32-bit header] [Data bits N] [Silence 0.25s]

Frequencies:
  SYNC preamble:  900 Hz (continuous tone)
  Header bit=1:   900 Hz
  Header bit=0:  1500 Hz
  Data bit=0:    1500 Hz  (freq_zero)
  Data bit=1:    2700 Hz  (freq_one)
```

Why these frequencies:

- Both 1500 Hz and 2700 Hz are within the passband of AAC at 128 kbps (20 Hz – 16 kHz)
- Both survive Opus at 32 kbps (400 Hz – 7700 Hz)
- 1200 Hz gap between them prevents cross-frequency confusion even under heavy compression

Why the embedded 32-bit header:
A naive sync-detection approach would search for the end of the preamble tone using FFT magnitude, but harmonics of the data tones (1500/2700 Hz) bleed into the 900 Hz bin, causing false positives. By embedding the exact bit count (`N`) as a header, we eliminate sync detection entirely — the decoder always knows exactly where the data starts and how many bits to read.

Running Tests 🧪

```
python full_test.py
```

Expected output:

```
============================================================
  🧪  SocialDisk v2.0 — Full Test Suite
============================================================

📦  [1/5] VaultCore — pack/unpack roundtrip
   ✅ Passed  Data matches 100%
   📊  Payload bits: 2,664

📸  [2/5] DCT Image — Instagram (simulated JPEG q=70)
   📊  Dimensions: 1080×1080  |  Usage: 48.07%  |  Delta: 50.0
   ✅ Passed  Recovered after JPEG q=70

💬  [3/5] DCT Image — WhatsApp (q=70, redundancy=5)
   📊  Dimensions: 1200×1200  |  Redundancy: 5×  |  Delta: 55.0
   ✅ Passed  Recovered after WhatsApp compression

🎬  [4/5] HyperGrid Video — 10 KB random file
   📊  Resolution: 1080×1920  |  Frames: 23  |  Duration: 0.77s
   ✅ Passed  SHA-256 orig=a1c463bf...  recv=a1c463bf...

♪  [5/5] Acoustic FSK — Encode/decode audio
   📊  Duration: 19.0s  |  Rate: 150 baud  |  Bits: 2,664
   ✅ Passed  Audio data matches perfectly

------------------------------------------------------------
  Result: 5/5 — SocialDisk v2.0 — 100% efficiency ✨
============================================================
```

Project Structure 📁

```
SocialDisk/
│
├── socialdisk/                    # Core library package
│   ├── __init__.py                # Package init, version, exports
│   ├── core.py                    # VaultCore: AES-256-GCM + Reed-Solomon + L
ZMA
│   ├── platforms.py               # 14+ platform profiles and dataclasses
│   ├── api.py                     # SocialVault: high-level unified API
│   ├── i18n/                      # Internationalization (Arabic + English)
│   │   ├── __init__.py            # i18n manager & settings persistence
│   │   ├── ar.py                  # Arabic translations
│   │   └── en.py                  # English translations
│   └── engines/
│       ├── __init__.py
│       ├── image.py               # DCT steganography engine
│       ├── video.py               # HyperGrid video engine
│       └── audio.py               # Acoustic FSK modem engine
│
├── cli.py                         # Beautiful CLI (Rich UI, interactive + com
mand mode)
├── full_test.py                   # 5-engine comprehensive test suite
├── test_import.py                 # Quick import verification
├── check_envs.py                  # Environment diagnostic tool
├── SocialVault.py                 # Backward-compatibility wrapper
│
├── requirements.txt               # Python dependencies
├── setup.py                       # Package installation
├── pyproject.toml                 # Modern build config
│
├── .github/
│   ├── workflows/ci.yml           # GitHub Actions: 3 OS × 3 Python versions
│   └── ISSUE_TEMPLATE/
│       ├── bug_report.yml
│       └── feature_request.yml
│
├── README.md                      # This file (bilingual)
├── CONTRIBUTING.md                # Contribution guide
└── LICENSE                        # MIT License
```

FAQ ❓

Q: Does the hidden data survive posting on Instagram?

A: Yes — the Instagram profile is tuned specifically for Instagram's JPEG compression (quality ≈70). The Reed-Solomon ECC corrects the remaining bit errors.

Q: Can anyone detect that the image contains hidden data?
A: The changes are designed to be statistically imperceptible. However, a determined forensic analyst with the original image could compare them. SocialDisk is not meant as a perfect forensic tool — it prioritizes data survival over perfect stealthiness.

Q: What is the maximum data size I can hide?
A: Depends on the media file size and platform. For a 1080×1080 Instagram image, typical capacity is around **500–800 bytes** of original data (after compression, encryption, and ECC overhead). Video capacity is much higher — tens of kilobytes per second of video.

Q: Is the password stored anywhere?
A: Never. The password is used only to derive an AES-256 key (via PBKDF2-SHA256 with 100,000 iterations + a random 16-byte salt stored inside the payload). The salt is different for every encode operation.

Q: What if the receiver doesn't use SocialDisk?
A: They cannot recover the data. Without SocialDisk and the correct password, the hidden data is indistinguishable from normal media noise.

Q: Does it work on Windows / Linux / macOS?
A: Yes — fully cross-platform. Tested on all three via GitHub Actions CI.

Legal Disclaimer ⚠️

SocialDisk is intended for **privacy protection, security research, and educational purposes only**.

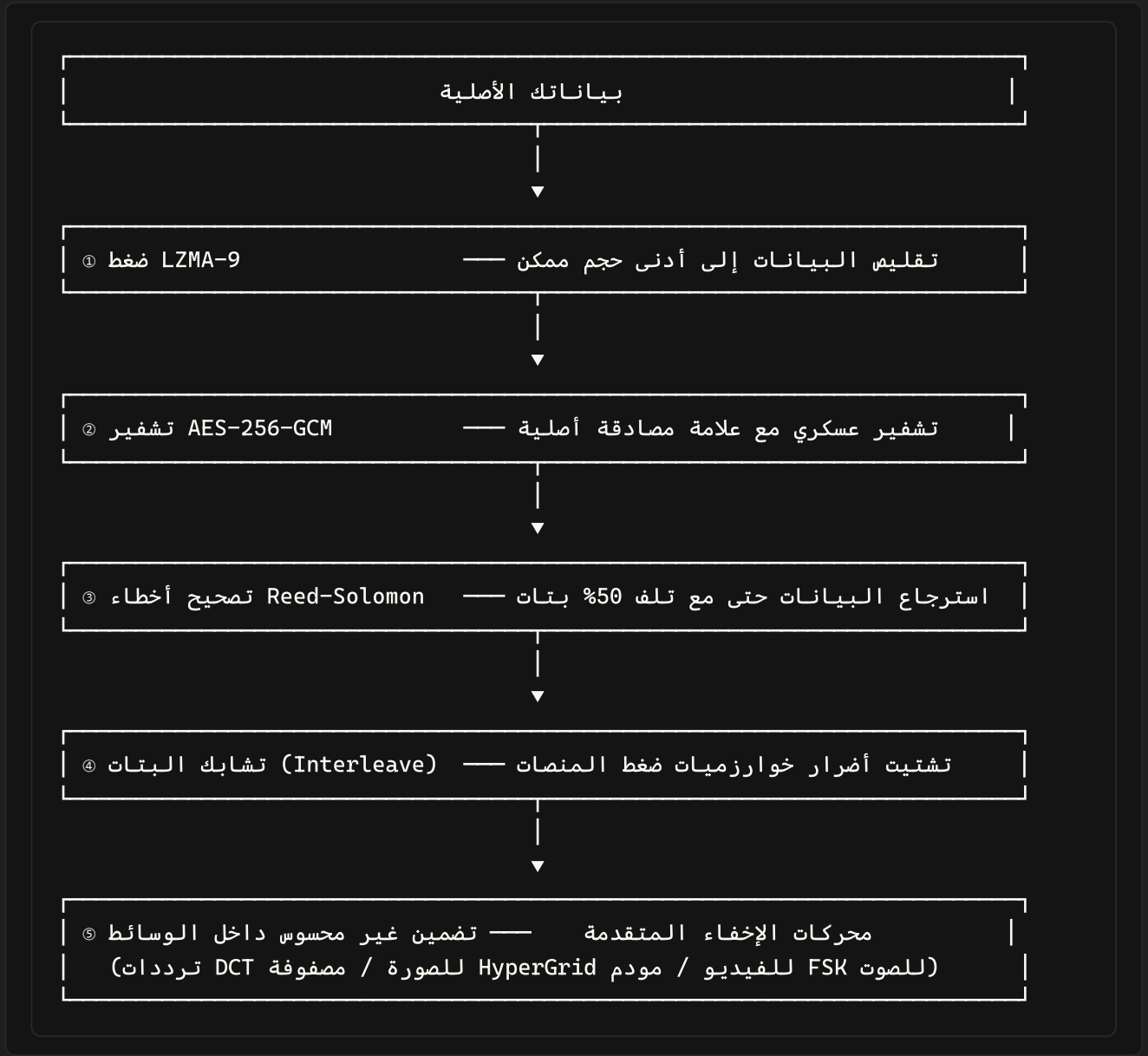
- .Do not use SocialDisk to violate the Terms of Service of any platform
- Do not use SocialDisk for illegal data transfer or to circumvent legitimate security .controls
- .The authors are not responsible for any misuse of this software
- .Use responsibly and at your own risk

SA العربية

📖 ما هو SocialDisk؟

هي أداة إخفاء وتشفير البيانات متعددة المنصات. تتيح لك **إخفاء أي ملف أو رسالة** داخل **SocialDisk** ملفات وسائط عادية — صور، فيديو، أو صوت — ثم نشرها على أي منصة تواصل اجتماعي. الشخص ويسترجع البيانات المخفية بشكل مثالي حتى بعد، SocialDisk على الطرف الآخر يُنزّل الوسيط، يُشغّل ضغط المنصة.

> كل بايت من البيانات يمر بـ **5 طبقات حماية** قبل الإخفاء:



✨ المميزات الكاملة

#	الميزة	الوصف
🔒	**تشفير AES-256-GCM**	تشفير عسكري مع علامة مصادقة — أي تعديل يُكتشف فوراً
🧬	**تصحيح أخطاء Reed-Solomon**	يستعيد البيانات حتى عند تلف 50% من البتات بسبب الضغط
🗜️	**ضغط LZMA-9**	أقصى إعداد لضغط LZMA يُطبَّق قبل التشفير
🔀	**تشابك البتات**	يبعثر البايتات عشوائياً (بمفتاح من كلمة المرور) لتوزيع الأضرار
🔑	**PBKDF2-SHA256**	كلمة المرور تُشتق بـ 100,000 تكرار + ملح عشوائي 16 بايت
📸	**محرك الصور DCT**	يُعدِّل معاملات JPEG الترددية — لا يُرى بالعين البشرية
🎬	**محرك الفيديو HyperGrid**	يضمّن البيانات في شبكة بكسل بصرية — يصمد أمام H.264
🎵	**محرك الصوت Acoustic FSK**	يُحوّل البيانات إلى نغمات صوتية — يصمد أمام AAC وOpus
🌍	**14+ منصة**	ملف إعداد مُعايَر لكل منصة بناءً على خصائص ضغطها
🖥️	**تفاعلية جميلة CLI**	قائمة بألوان وجداول وأشرطة تقدم (مكتبة Rich)
⌨️	**وضع سطر الأوامر**	قابل للأتمتة والتكامل مع سكريبتات أخرى
🐍	**Python API**	كلاس `SocialVault` للتكامل المباشر في كودك

🌍 المنصات المدعومة

المنصة	📸 صورة	🎬 فيديو	🎵 صوت
إنستغرام	✅	✅	✅
واتساب	✅	✅	✅
تيك توك	—	✅	✅
يوتيوب	—	✅	✅
تويتر / X	✅	✅	—
ديسكورد	✅	✅	✅
تيليغرام	✅	✅	✅
فيسبوك	✅	✅	—
لينكدإن	✅	—	—
سناب شات	✅	✅	—
سيغنال	✅	—	✅
ويشات	✅	—	—
ريدت	✅	—	—
iMessage	✅	—	✅

🚀 التثبيت

```
# استنساخ المستودع
git clone https://github.com/YOUR_USERNAME/SocialDisk.git
cd SocialDisk

# تثبيت المتطلبات
pip install -r requirements.txt

# التحقق من التثبيت
python test_import.py    # يجب أن يظهر: All imports OK ✅
python full_test.py      # يجب أن يظهر: 5/5 tests passed ✅
```

🖥️ طريقة الاستخدام — الوضع التفاعلي (للمبتدئين)

```
python cli.py
```

تظهر قائمة تفاعلية كاملة تُتيح لك اختيار كل شيء بسهولة.

⌨️ طريقة الاستخدام — سطر الأوامر

إخفاء ملف داخل صورة

```
python cli.py hide \
  --platform instagram \
  --type image \
  --input  سري.txt \
  --cover  صورة.jpg \
  --output مخفي.jpg \
  --password "كلمة_المرور"
```

استخراج ملف من صورة

```
python cli.py extract \
  --platform instagram \
  --type image \
  --input  مخفي_بعد_التنزيل.jpg \
  --output مسترجع.txt \
  --password "كلمة_المرور"
```

إخفاء ملف داخل فيديو

```
python cli.py hide --platform whatsapp --type video \
  --input ملف.pdf --cover مقطع.mp4 --output مخفي.mp4 \
  --password "كلمة_المرور"
```

إخفاء رسالة في صوت

```
python cli.py hide --platform discord --type audio \
  --input رسالة.txt --output صوت_مشفر.wav \
  --password "كلمة_المرور"
```

عرض جميع المنصات

```
python cli.py --platforms
```

Python API

```
from socialdisk import SocialVault

# إخفاء في صورة
vault = SocialVault("instagram")
vault.hide_in_image("صورة.jpg", "مخفي.jpg", open("سري.pdf","rb").read(), "كلمة_المرور")

# استخراج من صورة
data = vault.recover_from_image("مخفي.jpg", "كلمة_المرور")

# إخفاء في فيديو
vault2 = SocialVault("whatsapp")
vault2.file_to_video("مخفي.mp4", open("سري.zip","rb").read(), "كلمة_المرور")
data2 = vault2.video_to_file("مخفي.mp4", "كلمة_المرور")

# إخفاء في صوت
vault3 = SocialVault("discord")
vault3.text_to_audio("صوت.wav", "كلمة_المرور" ,"رسالتي السرية")
text = vault3.audio_to_text("صوت.wav", "كلمة_المرور")
```

تشغيل الاختبارات

```
python full_test.py
```

```
النتيجة: 5/5 اختبار نجح
🎉  SocialDisk v2.0 — 100% كفاءة ✨
```

أسئلة شائعة

س: هل البيانات تصمد فعلاً بعد رفع الصورة على إنستغرام؟

ج: نعم — ملف إعداد إنستغرام مُعايَر خصيصاً لضغط JPEG (جودة ≈70) الذي يُطبِّقه إنستغرام.

س: ما الحد الأقصى لحجم البيانات القابلة للإخفاء؟

ج: لصورة 1080×1080 على إنستغرام: حوالي **500–800 بايت** من البيانات الأصلية. للفيديو: عشرات الكيلوبايتات لكل ثانية فيديو.

س: هل كلمة المرور تُخزَّن في أي مكان؟
ج: لا أبداً. كلمة المرور تُستخدم فقط لاشتقاق مفتاح AES-256 عبر PBKDF2 مع ملح عشوائي مختلف في كل عملية.

س: هل يعمل على Windows / Linux / macOS؟
ج: نعم — متعدد المنصات بالكامل، مُختبَر على الثلاثة عبر GitHub Actions.

Made with ❤️ — SocialDisk v2.0.0

⬆ Back to top / العودة للأعلى